

Demonstrating Test Instance Lifecycle Change

- We use the Calculator project to demonstrate test instance lifecycle behavior.
- The example test class runs test methods in a specific order.
- By default, JUnit creates a new test instance for each test method.
- This default lifecycle is known as **PER_METHOD**.

 **by Rodrigo Martins Pagliares**

Last update: 12/26/2025



Back to our Calculator project

The screenshot shows an IDE window for a project named "CalculatorMavenProject". The left sidebar displays the project structure, with the "test" directory expanded. The "test" directory contains a "java" subdirectory, which in turn contains an "org.example" package. This package contains several test classes, including "MethodsOrderedByOrderIndexTest", which is currently selected. The main editor area shows the code for "MethodsOrderedByOrderIndexTest.java". The code is a Java class with three test methods: "testB()", "testA()", and "testD()", each annotated with "@Test" and "@Order". The methods are ordered 4, 2, and 3 respectively. The status bar at the bottom indicates the file is at line 25, column 13, using LF line endings, UTF-8 encoding, and 4 spaces for indentation.

```
5  @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
6  public class MethodsOrderedByOrderIndexTest {
7      @Order(4)
8      @Test
9      public void testB(){
10         System.out.println("Executing testB");
11     }
12
13     @Order(2)
14     @Test
15     public void testA(){
16         System.out.println("Executing testA");
17     }
18
19     @Order(3)
20     @Test
21     public void testD(){
22         System.out.println("Executing testD");
23     }
24
25     @Order(1)
26     @Test
27     public void testC(){
28         System.out.println("Executing testC");
29     }
```



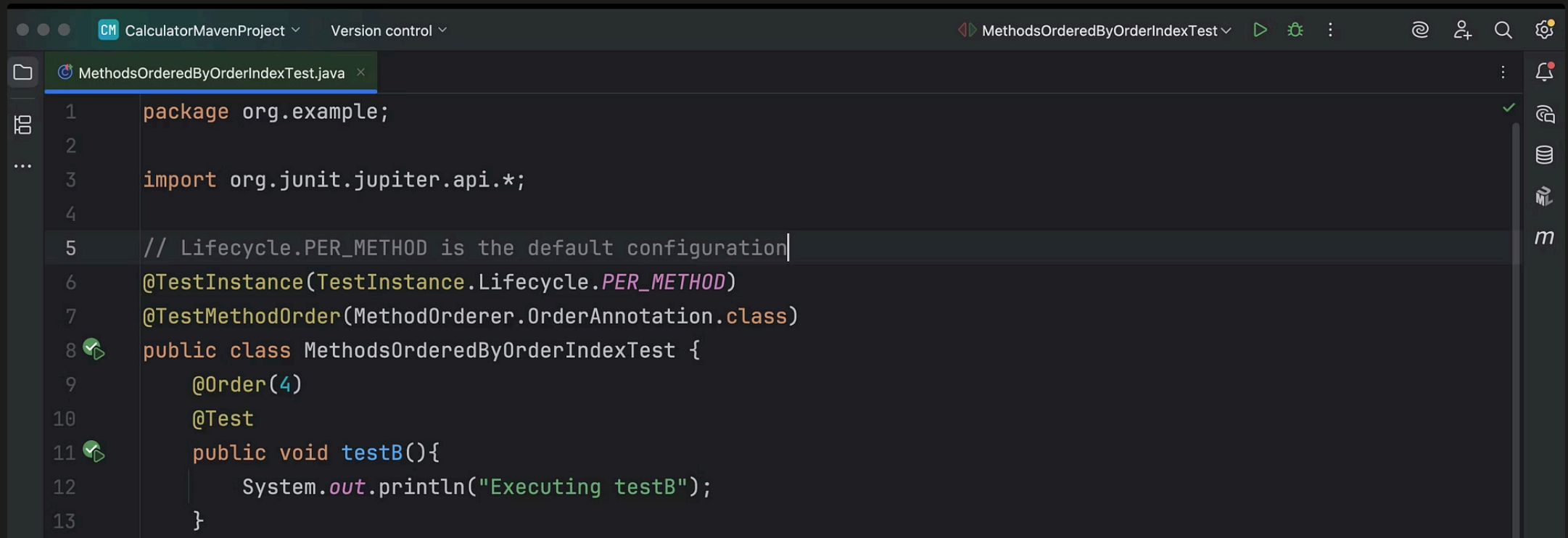
Default Lifecycle: PER_METHOD

- `PER_METHOD` means **one test class instance per test method**.
- This **behavior** is equivalent to using the **default** `TestInstance` **configuration**.
- **Each test method starts with a fresh object** state.
- **Instance variables** are **reset** for **every test execution**.

Default Lifecycle: PER_METHOD (contd.)

Line 6 configures the **test instance lifecycle**.

Since `Lifecycle.PER_METHOD` is the default, this setting can be omitted.

A screenshot of an IDE window showing a Java file named 'MethodsOrderedByOrderIndexTest.java'. The code is as follows:

```
1 package org.example;
2
3 import org.junit.jupiter.api.*;
4
5 // Lifecycle.PER_METHOD is the default configuration
6 @TestInstance(TestInstance.Lifecycle.PER_METHOD)
7 @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
8 public class MethodsOrderedByOrderIndexTest {
9     @Order(4)
10    @Test
11    public void testB(){
12        System.out.println("Executing testB");
13    }
```

The IDE interface includes a top bar with project and version control information, a left sidebar with a file explorer, and a right sidebar with various tool icons. The code is color-coded, and line numbers are visible on the left.

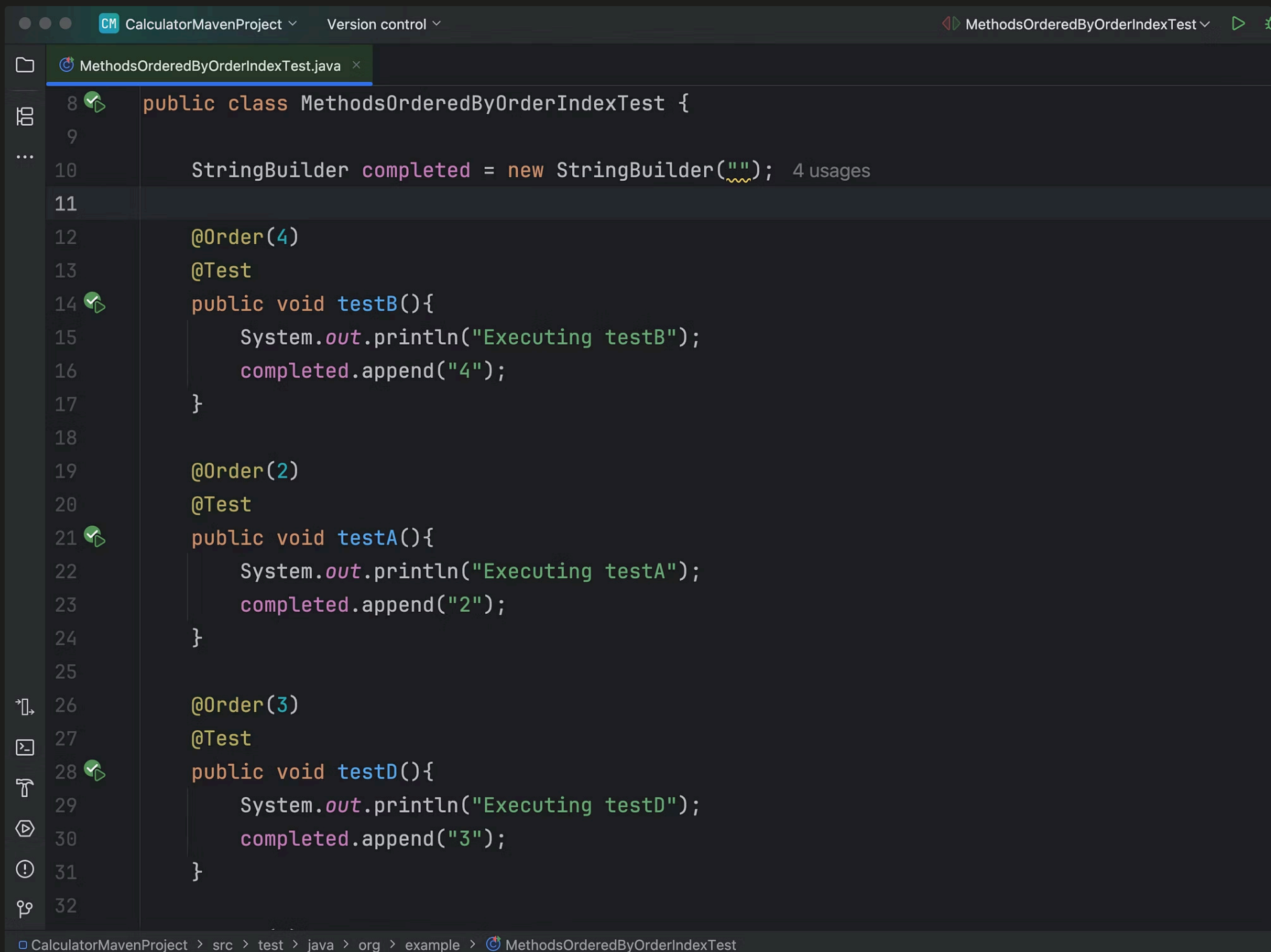
Example: Using an Instance Variable to Observe Behavior



- An instance variable of type `StringBuilder` is added to the test class.
- Each **test method appends** a different **value** to this **variable**.
- The variable starts empty when the test instance is created.
- Its content reveals whether state is shared or isolated.

Example: Using an Instance Variable to Observe Behavior (contd.)

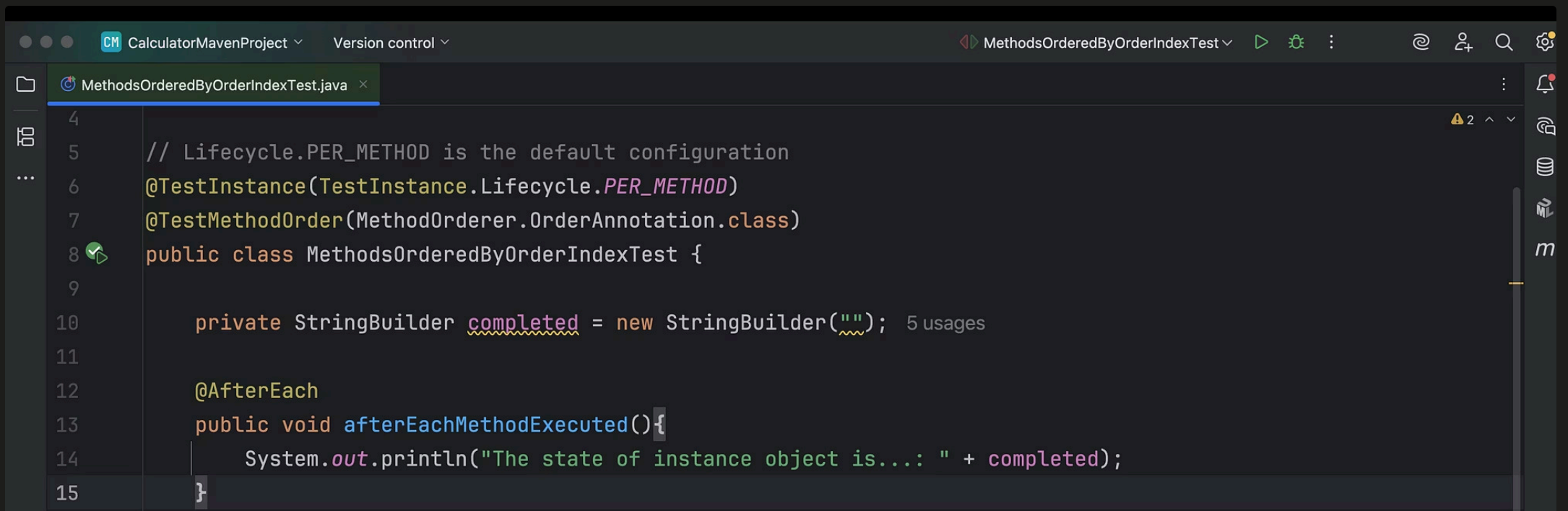
To demonstrate changing the **test instance Lifecycle**, lets use a **shared instance variable (Line 10)** of the type `StringBuilder`.



The screenshot shows an IDE window for a project named "CalculatorMavenProject". The active file is "MethodsOrderedByOrderIndexTest.java". The code defines a class with three test methods, each annotated with `@Order` and `@Test`. A shared instance variable `completed` of type `StringBuilder` is declared at line 10. The methods `testB`, `testA`, and `testD` append the numbers 4, 2, and 3 respectively to the `completed` variable. The IDE interface includes a sidebar with icons for Explorer, Search, Run and Debug, and other tools. The bottom status bar shows the file path: "CalculatorMavenProject > src > test > java > org > example > MethodsOrderedByOrderIndexTest".

```
8  public class MethodsOrderedByOrderIndexTest {
9
10     StringBuilder completed = new StringBuilder(""); 4 usages
11
12     @Order(4)
13     @Test
14     public void testB(){
15         System.out.println("Executing testB");
16         completed.append("4");
17     }
18
19     @Order(2)
20     @Test
21     public void testA(){
22         System.out.println("Executing testA");
23         completed.append("2");
24     }
25
26     @Order(3)
27     @Test
28     public void testD(){
29         System.out.println("Executing testD");
30         completed.append("3");
31     }
32 }
```

Example: Using an Instance Variable to Observe Behavior (contd.)



```
4
5 // Lifecycle.PER_METHOD is the default configuration
6 @TestInstance(TestInstance.Lifecycle.PER_METHOD)
7 @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
8 public class MethodsOrderedByOrderIndexTest {
9
10     private StringBuilder completed = new StringBuilder(""); 5 usages
11
12     @AfterEach
13     public void afterEachMethodExecuted(){
14         System.out.println("The state of instance object is...: " + completed);
15     }
```

Printing State After Each Test (**Lines 12-15**)

@AfterEach Lifecycle Method

An **@AfterEach** lifecycle method is added.

Prints Current State

This method prints the current state of the instance variable.

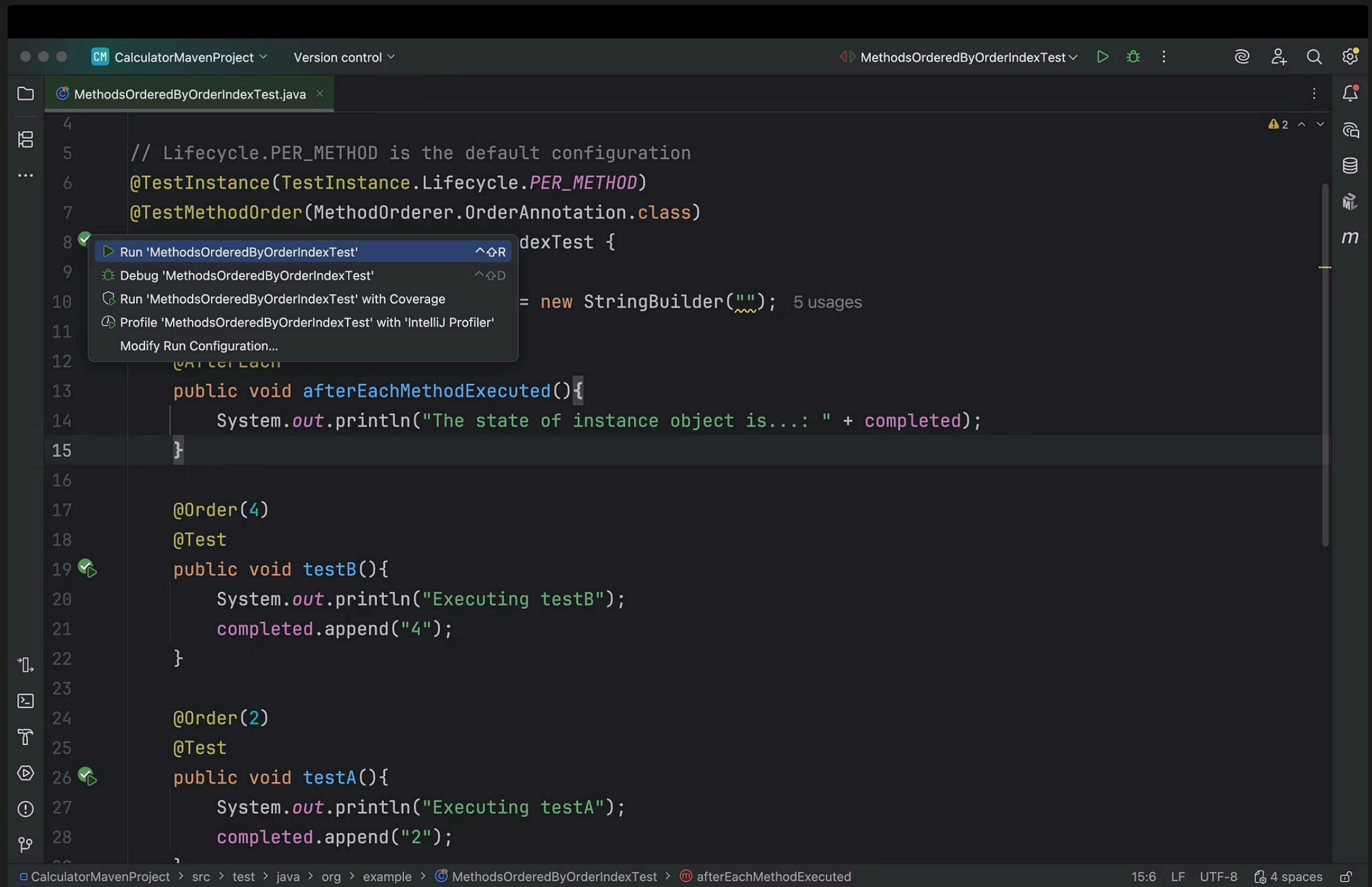
Runs After Every Test

It runs after every test method execution.

Visualizes Behavior

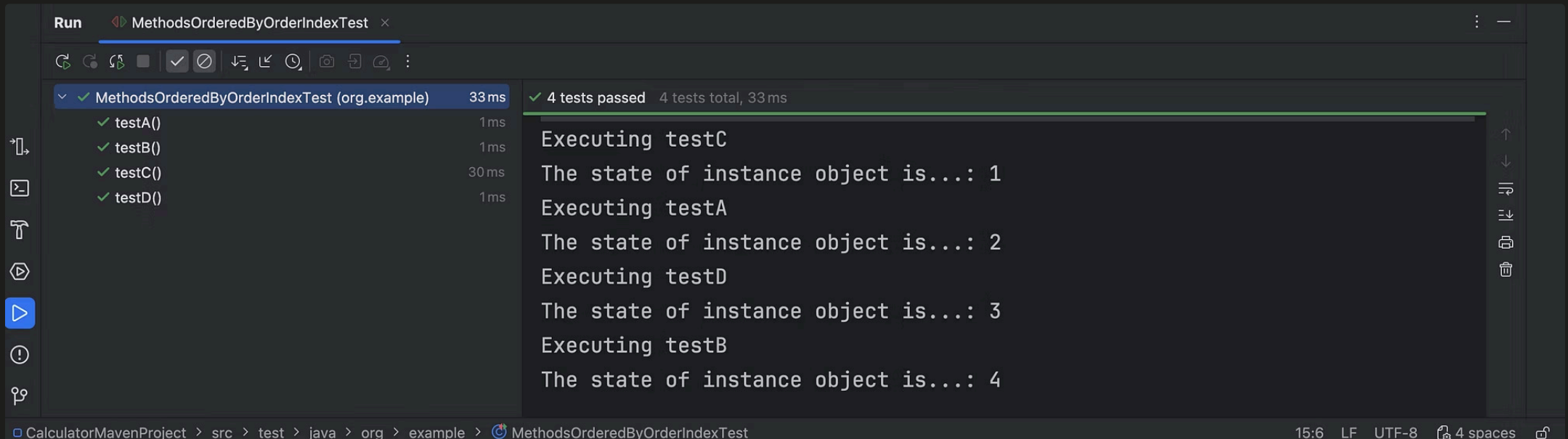
The output helps visualize instance lifecycle behavior.

Run all tests!



Results with PER_METHOD Lifecycle

- Each test prints **only one appended value**.
- The **instance variable** contains a **single character every time**.
- This **confirms** a **new test instance** is created **per method**.
- **No state** is **shared across test methods**.



The screenshot shows the Run window of an IDE. The top bar indicates the test class is `MethodsOrderedByOrderIndexTest`. Below the toolbar, a table lists the test methods and their execution times:

Test Method	Execution Time
✓ testA()	1 ms
✓ testB()	1 ms
✓ testC()	30 ms
✓ testD()	1 ms

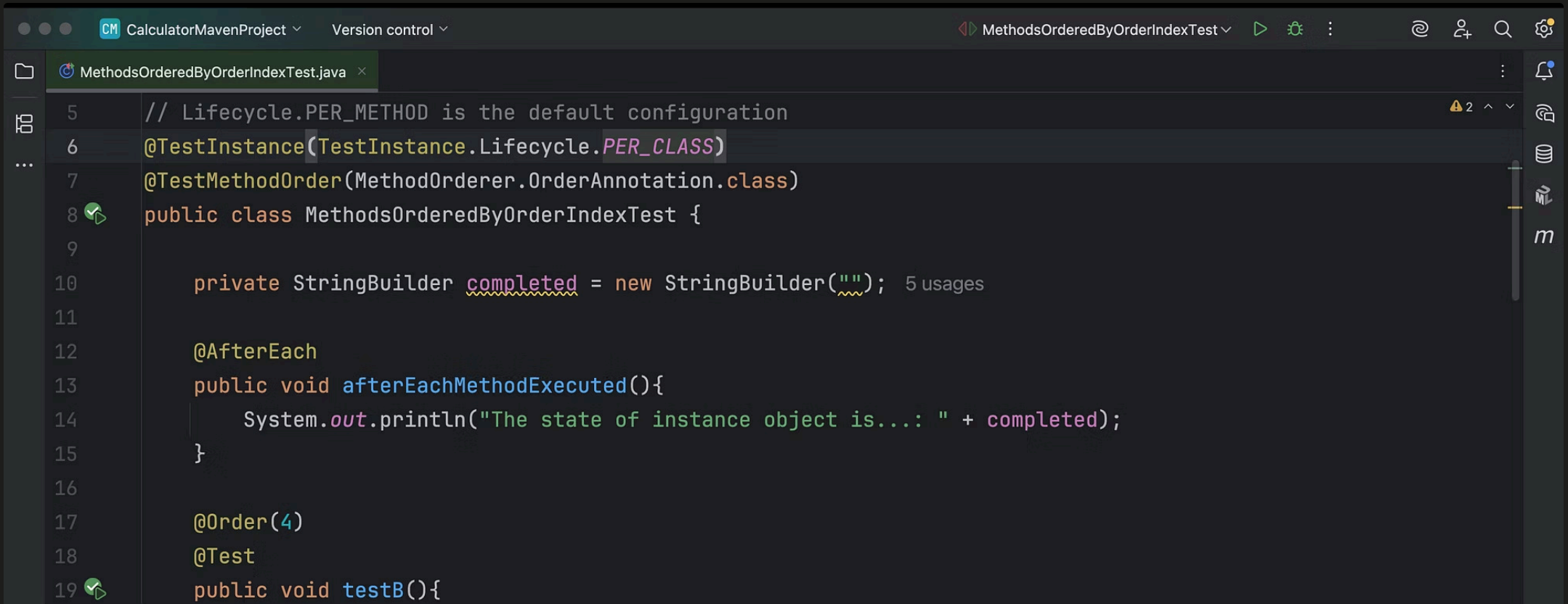
To the right of the table, a summary states: **✓ 4 tests passed** 4 tests total, 33 ms. The output pane on the right shows the following log:

```
Executing testC
The state of instance object is...: 1
Executing testA
The state of instance object is...: 2
Executing testD
The state of instance object is...: 3
Executing testB
The state of instance object is...: 4
```

The bottom status bar shows the file path: `CalculatorMavenProject > src > test > java > org > example > MethodsOrderedByOrderIndexTest`, along with encoding and formatting details: 15:6 LF UTF-8 4 spaces.

Results with PER_CLASS Lifecycle

- The **test class** is annotated with **@TestInstance(PER_CLASS)** - Line 6.
- Only **one test instance** is created for the **entire class**.
- All **test methods execute** on the **same object instance**.
- **Instance variables persist across test executions**.



```
5 // Lifecycle.PER_METHOD is the default configuration
6 @TestInstance(TestInstance.Lifecycle.PER_CLASS)
7 @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
8 public class MethodsOrderedByOrderIndexTest {
9
10     private StringBuilder completed = new StringBuilder(""); 5 usages
11
12     @AfterEach
13     public void afterEachMethodExecuted(){
14         System.out.println("The state of instance object is...: " + completed);
15     }
16
17     @Order(4)
18     @Test
19     public void testB(){
```

Results with PER_CLASS Lifecycle

01

Variable Accumulates Values

The instance variable accumulates values across tests.

02

Second Test Sees Previous Data

Second test sees data from the first test.

03

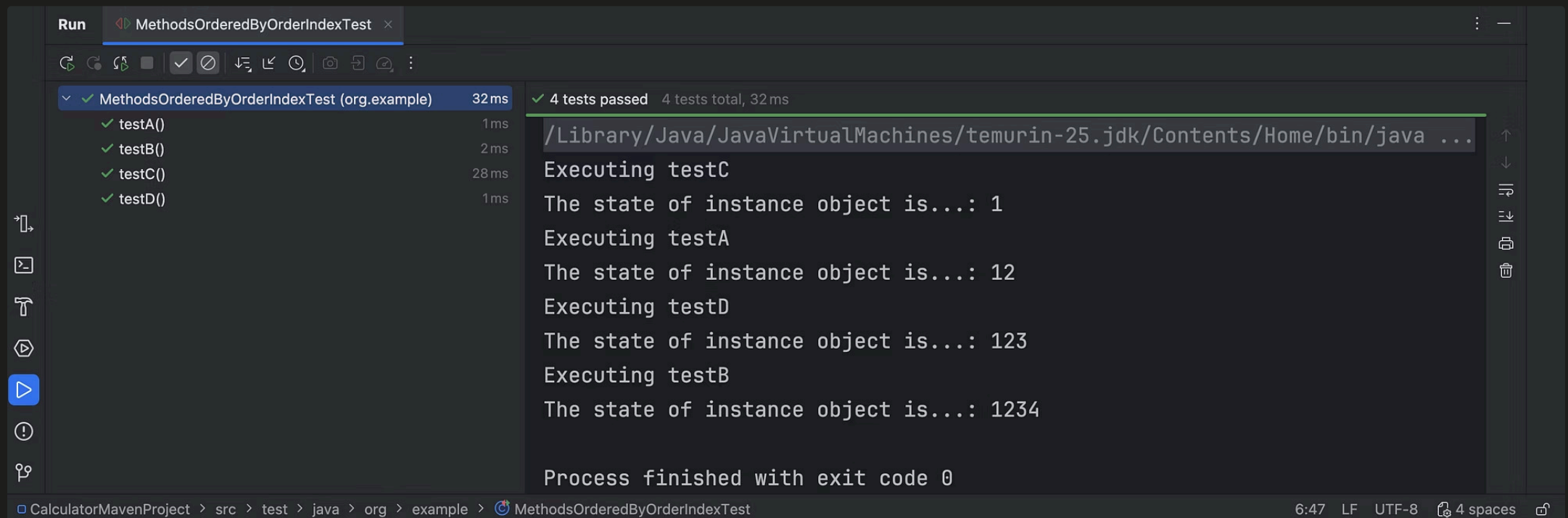
Final Test Shows All Values

Final test shows all appended values together.

04

Confirms Shared State

This confirms shared state across test methods.



The screenshot shows an IDE's Run window for a test class named `MethodsOrderedByOrderIndexTest`. The test suite consists of four methods: `testA()`, `testB()`, `testC()`, and `testD()`. All four tests passed successfully. The execution order is `testC`, then `testA`, then `testD`, and finally `testB`. The output shows that the state of the instance object is shared across all tests, with values being appended: 1 (from testC), 12 (from testA), 123 (from testD), and 1234 (from testB). The process finished with exit code 0.

```
Run | MethodsOrderedByOrderIndexTest x
✓ 4 tests passed 4 tests total, 32ms
✓ testA() 1ms
✓ testB() 2ms
✓ testC() 28ms
✓ testD() 1ms

/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...
Executing testC
The state of instance object is...: 1
Executing testA
The state of instance object is...: 12
Executing testD
The state of instance object is...: 123
Executing testB
The state of instance object is...: 1234

Process finished with exit code 0
```



Conclusion

PER_METHOD Ensures Isolation

`PER_METHOD` ensures isolation by default in JUnit.

PER_CLASS Allows Shared State

`PER_CLASS` allows shared state across test methods.

Lifecycle Choice Affects Behavior

Instance lifecycle choice directly affects test behavior.

Use PER_CLASS Carefully

`PER_CLASS` is useful but must be used carefully.



Let's Discuss

Open Floor Discussion

- When is shared state between test methods justified?
- How can `PER_CLASS` introduce hidden test dependencies?
- Should `PER_CLASS` be limited to integration testing scenarios?